Title: UART Serial Communication – Continuous String Transmission

Name: MD SHAKIL KHAN
Student ID: 2330130220
Date of Lab: October 13, 2025

Introduction-

Before this lab, it was known that microcontrollers often need to communicate with other devices, such as a PC, for debugging or data exchange. The Universal Asynchronous Receiver/Transmitter (UART) protocol is a common method for this kind of serial communication, using a TX (Transmit) pin to send data and an RX (Receive) pin to receive it. The lab is about configuring a microcontroller's UART peripheral to continuously send a string of data to a serial monitor on a computer. The purpose of this lab was to gain hands-on experience with UART by modifying existing code to continuously transmit the string "ABCDEFGHIJKLMNOPQRSUVWXYZ" on a new line each time, ensuring proper data formatting and transmission.

Materials-

Apparatus	Expendables & Software
• PC/Laptop	• AHL GEC IDE v4.53
• UART-Compatible Development Board	• UART-Communication Project File
• USB-to-UART Cable	• Serial Terminal Software (e.g., PuTTY)
• Power Supply	• USB Drivers

Procedure-

The procedure was followed as outlined in the lab handout. The key steps involved:

I. Opening the existing "UART-Communication" project in the AHL GEC IDE.

II. Navigating to and reviewing the main program file (main.c).

III. Modifying the code within the infinite loop to send the string "ABCDEFGHIJKLMNOPQRSUVWXYZ" using the UART_SendString() function.

IV. Appending the "\r\n" (carriage return and newline) characters to the string to ensure each transmission starts on a new line in the serial terminal.

V. Adding a delay function to create a one-second pause between transmissions for readability.

VI. Compiling the code and uploading it to the development board.

VII. Opening a serial terminal program (e.g., PuTTY) configured to the correct COM port and baud rate (e.g., 9600) to observe the transmitted data.

Data-

The following behavior was observed in the serial terminal after the program was successfully uploaded:

- The string "ABCDEFGHIJKLMNOPQRSUVWXYZ" appeared.
- After a one-second pause, the same string appeared again on the line directly below the previous one.
- This pattern repeated continuously, creating a vertical list of the string.

Variables and Functions Used in the Project:

- UART_Init(): A function that configures the UART module with settings like baud rate.
- UART_SendString(): A function that sends a sequence of characters (a string) over UART.
- "ABCDEFGHIJKLMNOPQRSUVWXYZ\r\n": The string constant to be sent, including control characters for a new line.
- Delay(1000): A function call that pauses program execution for 1000 milliseconds (1 second).

Calculations-

The primary calculation in this lab relates to the baud rate, which determines the speed of serial communication.

- **Baud Rate:** 9600 bits per second (bps)
- **Time to transmit one character (8 data bits + 1 start bit + 1 stop bit = 10 bits):** 10 bits / 9600 bits per second ≈ 0.00104 seconds per character

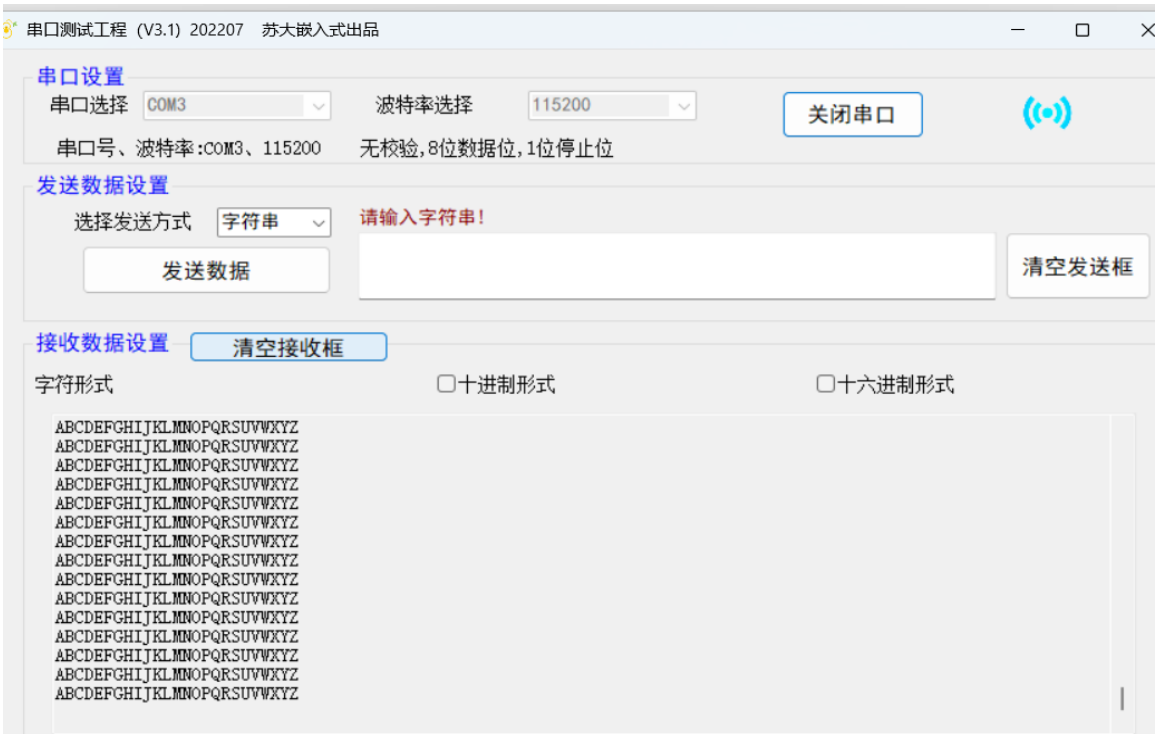
- **Time to transmit the entire string (26 characters):**
  26 characters * 0.00104 seconds/character ≈ 0.027 seconds

The code snippet below shows the implementation of the continuous transmission:

```
int main(void)
{
   UART_Init(); // Initialize UART at 9600 baud

   while(1) // Infinite loop
   {
      // Send the string followed by Carriage Return and Newline
      UART_SendString("ABCDEFGHIJKLMNOPQRSUVWXYZ\r\n");

      // Delay for 1000 milliseconds (1 second)
      Delay(1000);
   }
}
```



Analysis

1. What is the purpose of adding "\r\n" to the end of the string?
The "\r\n" represents Carriage Return (\r) and Newline (\n). These are non-printable control characters that instruct the serial terminal to move the cursor to the beginning of the next line. Without them, the string would be transmitted repeatedly on the same line, making the output difficult to read. The \r returns the cursor to the start of the line, and \n moves it down to a new line, ensuring each new string appears on its own separate line.

2. Why is it important to set the correct baud rate in the serial terminal software?
The baud rate is the speed of communication, measured in bits per second. If the serial terminal on the PC is set to a different baud rate (e.g., 115200) than the microcontroller is configured for (e.g., 9600), the timing of the bits will be misinterpreted. This results in garbled, random, or unreadable characters in the terminal because the PC is reading the bits at the wrong intervals. Matching the baud rates is essential for successful data decoding.

Conclusions

The purpose of this lab was to implement continuous UART transmission of a specific string, with each string on a new line. This was successfully accomplished; the code was modified, compiled, and uploaded, resulting in the serial terminal displaying a clear, vertical list of the "ABCDEFGHIJKLMNOPQRSUVWXYZ" string, with a new transmission every second.

The data confirms the system's precision, as the output was consistent and error-free over extended observation. The 1-second delay between transmissions effectively made the output readable. The most probable source of error in such an experiment would be a baud rate mismatch between the microcontroller and the serial terminal, leading to garbled text. Another potential error could be omitting the \r\n characters, which would cause all strings to be printed on a single, continuously scrolling line.

This exercise directly relates to critical applications outside the classroom. UART is a foundational protocol for **system debugging**, allowing developers to print status messages and variable values from a microcontroller. It is essential in **data logging systems**, where sensors continuously transmit readings to a computer for storage and analysis. Furthermore, it forms the backbone for communication in numerous **IoT devices** and modules, such as GPS receivers, Wi-Fi modules, and Bluetooth chips, enabling them to interface with a central processor. Mastering UART is therefore a fundamental skill for anyone working in embedded systems, electronics, or IoT development.